The Universal Proof-Horizon Operator

From MIU and the nth Well-Formed Formula
to ZFC, Predator, and Automated Logical Deciders

Brian Tenneson

M.A. Mathematics; M.S. Applied Data Science

Research synthesis and manuscript development assisted by OpenAI's ChatGPT

Version 1.0 — August 3, 2026

Attribution and intellectual provenance.

Brian Tenneson originated the research question and developed the core synthesis. In particular, he connected Matos's constructive work on MIU, effective enumeration of well-formed formulas by Gödel coding, shortest-proof search, his SIC Proof-Horizon framework, theorem-space growth, Predator, and automated logical deciders. AI assistance helped recover, organize, test, criticize, and express the argument. The mathematical direction and the decisive conceptual moves came primarily from Tenneson; the AI served as an intensive research and editorial collaborator.

Novelty disclaimer.

The computability fact proved in this monograph — that a shortest proof can be recovered by a partial algorithm once the proof system and a suitable effective size measure are fixed — is classical in substance. No claim is made that recursive enumerability of formal proofs is new. The contribution offered here is the explicit theorem statement, the careful boundary conditions, and the historical and architectural synthesis linking a 2016 lecture, MIU, OEIS A331536, semi-ideal computers, machine-guided proof search, Predator, and automated logical deciders.

Abstract

A formal theory such as ZFC has an effectively recognizable syntax and an effectively checkable proof relation. Once a Gödel numbering is fixed, its well-formed formulas can be listed as $\varphi_0, \varphi_1, \ldots$. This monograph isolates a precise consequence: relative to a fixed proof calculus and a computable proof-cost measure with effectively finite sublevel sets, there is a partial computable operator that takes n and returns a canonical shortest proof of φ_n whenever φ_n is a theorem. It diverges when no proof exists. The theorem therefore does not decide ZFC, evade Church's undecidability theorem, or overcome Gödel incompleteness. It identifies the exact positive result left standing by those barriers: proof recovery is universal on theorems, while termination on non-theorems is not guaranteed.

The result is placed in a ten-year research arc. Douglas Hofstadter's MIU system provided a small formal universe; Armando Matos characterized its theorems and gave proof-producing algorithms; Tenneson's OEIS A331536 quantified the explosive growth of its reachable theorem space; Tenneson's SIC Proof-Horizon Theorem converted shortest proof length into a machine stage; and Predator replaces indiscriminate enumeration with learned and creative navigation. Dovetailing searches for a conjecture and its negation yields the primitive core of an automated logical decider, provided that silence is reported as UNKNOWN rather than mistaken for refutation or independence.

Keywords: automated theorem proving, proof theory, shortest proof, Gödel numbering, ZFC, MIU, OEIS A331536, semi-ideal computer, proof horizon, machine learning, Predator, automated logical decider, conjecture settlement.

Contents

List of Figures

Reader's Guide

The central mathematical result is Theorem 5.2. Readers who want only the theorem and its proof may begin with Chapter 5. The historical and conceptual path begins in Chapter 1. The practical consequences for automated theorem proving appear in Chapters 9 and 10. The limitations, including the distinction between proof-size and line-count minimality, are collected in Chapter 7.

Chapter 1

A Question Asked in 2016

In 2016 Brian Tenneson presented a lecture titled *Proof Theory and Automated Theorem Proving* to graduate mathematics students at the University of Montana [1]. According to the author's retrospective account, the lecture was not merely about whether a computer could verify a formal derivation. It was animated by a harder question: could a machine move through a formal system in a way that deserved to be called intelligent?

The question grew naturally from Hofstadter's MIU system [2]. MIU is tiny: an alphabet, one axiom, and four string-rewriting rules. Yet the space of reachable expressions expands rapidly, and the famous target `MU` cannot be reached. A machine can apply rules mechanically, but a useful theorem prover must somehow decide which legal move deserves attention.

Ten years later, the same question has become an engineering program. Large language models can compare descriptions, recover analogies, write code, and coordinate information distributed across many documents. Symbolic automated theorem provers can search formal states. Independent verifiers can decide whether a proposed certificate is correct. Predator attempts to combine learned guidance, exploratory policies, and formal checking. Automated logical deciders extend the architecture by searching symmetrically for proof, refutation, and eventually independence certificates.

> *"This is my dream come true. The LLM helping me reconnect the lecture I gave in 2016, my OEIS entry, and the prover running now is exactly the kind of machine I was dreaming about."*
>
> — *Brian Tenneson, retrospective note, August 3, 2026*

1.1 Analogy as a bridge from unknown to known

Hofstadter and Sander argue that analogy is central to cognition [3]. Tenneson's informal "tag" vocabulary gives a compact computational picture of one aspect of that view. Suppose a listener does not yet know Superman but already associates lightning with a feature tag such as

$$\langle \mathsf{quick} \rangle.$$

The statement "Superman is fast as lightning" aligns the unknown object with a known object along that feature. The new representation is incomplete — speed is only one property, and the comparison may be figurative rather than literal — but it has converted one dimension of Superman from unknown to partially known.

A modern language model has an enormous supply of possible "other ends" for such alignments: texts, concepts, metaphors, mathematical structures, algorithms, and examples. That makes it useful for proposing paths through conceptual space. It does not, however, turn an analogy into a proof. The division of labor in this project is therefore deliberate:

analogy and language propose; formal search constructs; verification certifies.

1.2 Curry–Howard is related, but it answers a different question

The Curry–Howard correspondence identifies propositions with types and proofs with programs [4]. It helps explain what a proof can be as a computational object. The present research program asks a complementary question: how does an agent discover the right proof-object in an overwhelmingly large space? Curry–Howard supplies part of the ontology; proof search, analogy, heuristics, and learning supply navigation.

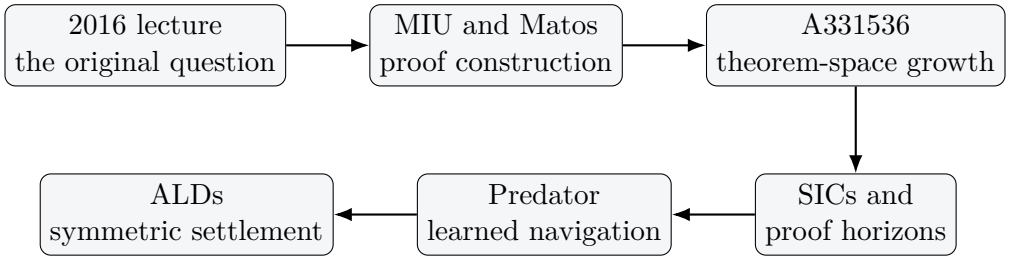

Figure 1.1: The research arc reconstructed in this monograph. The arrows indicate conceptual development, not claims that each later object was fully specified in 2016.

Chapter 2

MIU: A Small Formal World with a Large Search Problem

2.1 The formal system

Hofstadter's MIU system has alphabet $\{M, I, U\}$, axiom `MI`, and the following rules, where x and y are strings:

$$\begin{aligned} &\text{I.} && xI \longrightarrow xIU, \\ &\text{II.} && Mx \longrightarrow Mxx, \\ &\text{III.} && xIIIy \longrightarrow xUy, \\ &\text{IV.} && xUUy \longrightarrow xy. \end{aligned}$$

Every legal MIU theorem begins with M and is followed by a finite string of I's and U's. The system is pedagogically effective because syntax, theoremhood, search, invariants, and proof length can all be seen without a large logical apparatus.

2.2 Matos: characterization and proof production

Matos proved that an MIU well-formed formula Mx is a theorem exactly when the number of I's in x is not divisible by three, and he supplied an algorithm that outputs a standard proof for each theorem [5]. Matos and Antunes then analyzed shorter proofs and proved asymptotic bounds for proof length and total proof symbols [6]. Matos's subsequent study emphasized a distinction crucial to the present monograph: a system may have a simple characterization of its theorems while minimum-proof behavior remains nontrivial [7].

This is the first bridge to the ZFC question. In MIU, special algebraic structure permits a theorem-specific constructive algorithm. For a general effective theory, no comparable closed formula is promised. Nevertheless, a universal proof-recovery procedure exists by enumeration. Its price is divergence on non-theorems and catastrophic inefficiency.

2.3 A shortest proof is relative to a metric

Matos studies proof length in lines and in symbols. These are distinct objective functions. More generally, "the shortest proof" is incomplete unless one fixes:

(i) the formal theory and its axioms;

(ii) the proof calculus and allowed derived rules;

(iii) the encoding of formulas and proof objects;

(iv) the cost measure — lines, inference steps, symbols, bits, weighted steps, or another metric.

This dependence is a standard theme in proof complexity [8]. In the universal theorem below, the cost measure must also have effectively finite sublevel sets. Total encoded size is a canonical example. Line count alone needs additional finite-branching or bounded-syntax hypotheses; this issue is treated carefully in Chapter 7.

Chapter 3

A331536: Measuring Theorem-Space Explosion

Tenneson's OEIS entry A331536 records the number of distinct MIU theorems reachable within a bounded number of rule applications from `MI` [9]. With the OEIS indexing convention, $a(n)$ counts the theorems reachable in at most $n-1$ rule applications. The values currently recorded are

$$1, 3, 6, 11, 25, 69, 282, 1730, 15885, 210105, 3986987, 106053474, 3937200362.$$

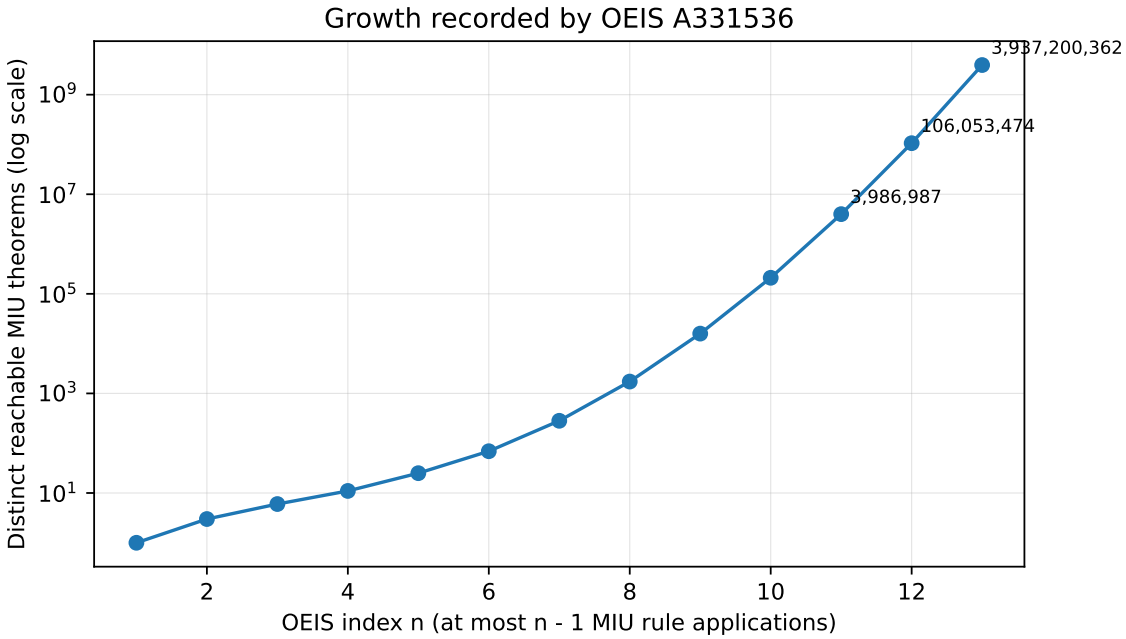

Figure 3.1: The growth of A331536 on a logarithmic vertical scale. By the thirteenth recorded index, the cumulative reachable set exceeds 3.9 billion distinct MIU theorems.

The sequence does not prove that every proof problem has this growth law, nor does it measure ZFC directly. It performs a more fundamental service: it makes visible how quickly even a four-rule system can overwhelm indiscriminate enumeration. The practical ATP question is therefore not whether legal consequences can be generated, but how a machine can avoid materializing almost all of them.

A331536 supplies the search problem.
The Proof-Horizon Operator supplies the universal baseline.
Predator attempts to supply navigation.

Chapter 4

From Gödel Numbers to the nth Well-Formed Formula

The phrase "invert Gödel numbers" is nearly right, but it needs one qualification. Not every natural number must code a valid expression. The correct construction decodes candidate numbers, filters for syntactic validity, and counts the survivors.

Definition 4.1 (Effective syntax)**.** *Let L be a recursively presented formal language. Fix an injective Gödel coding of finite strings by natural numbers. Assume there is a total computable predicate*

$$\mathsf{WFF}_L(m) = 1$$

exactly when m is the code of an L-well-formed formula, and a total computable predicate $\mathsf{Sent}_L(m)$ for sentences.

For the ordinary first-order language of set theory, parsing is decidable. Variables, parentheses, equality, membership, logical connectives, and quantifiers are finite syntactic objects, so a parser can determine whether a decoded string is a formula and whether it has free variables.

Proposition 4.2 (The nth formula and sentence)**.** *There are total computable one-to-one enumerations*

$$n \longmapsto \varphi_n \qquad \textit{and} \qquad n \longmapsto \sigma_n$$

of all L-well-formed formulas and all L-sentences, respectively.

Proof. Scan candidate codes $m = 0, 1, 2, \ldots$. Retain m exactly when $\mathsf{WFF}_L(m) = 1$. Because the predicate is decidable, the scan can count valid codes until the nth one is reached and then decode it. Injectivity of the Gödel coding prevents duplication. The sentence enumeration is identical with Sent_L in place of WFF_L. □

Thus there really is a computable meaning of "the nth ZFC formula" after the coding and ordering convention have been fixed. Different codings produce different enumerations, but every effective choice lists the same countable syntax.

Remark 4.3 (What this does not enumerate). The enumeration lists expressions, not truths. It includes theorems, refutable statements, independent statements, and malformed mathematical ideas that are nevertheless syntactically valid. Syntax is the input space; proof search supplies logical status when it can.

Chapter 5

The Universal Proof-Horizon Operator

5.1 The exact assumptions

The headlining theorem is most naturally stated for any effective proof system, with ZFC as a corollary.

Definition 5.1 (Effective proof presentation)**.** *An effective proof presentation is a triple* (T, V, c) *satisfying the following conditions.*

(1) T *is a formal theory in a recursively presented language.*

(2) $V(p, q)$ *is a total computable verifier. It returns* 1 *exactly when* p *codes a valid* T*-proof whose concluding sentence has code* q*.*

(3) $c(p) \in \mathbb{N}$ *is a total computable proof cost.*

(4) For every $b \in \mathbb{N}$*, the sublevel set*

$$P_b = \{p \in \mathbb{N} : c(p) \leq b\}$$

is finite and can be effectively listed.

The last condition will be called *effective properness*. Total symbol count, total bit length of a self-delimiting proof encoding, or the length of the proof file in bytes are natural examples. The condition ensures that "search every cheaper proof first" is an executable instruction rather than an idealized infinitary demand.

Fix the sentence enumeration $\sigma_0, \sigma_1, \ldots$ from Theorem 4.2. Order proof codes first by $c(p)$ and then numerically to break ties.

Theorem 5.2 (Universal Proof-Horizon Operator)**.** *For every effective proof presentation (T, V, c), there is a partial computable function*

$$H_{T,V,c} : \mathbb{N} \rightharpoonup \mathbb{N}$$

such that, for every n:

(i) $H_{T,V,c}(n)\downarrow$ if and only if $T \vdash \sigma_n$;

(ii) when defined, $H_{T,V,c}(n)$ is the least proof code among all T-proofs of σ_n having minimum c-cost;

(iii) consequently,

$$\mathrm{dom}(H_{T,V,c}) = \{n : T \vdash \sigma_n\}.$$

Proof. On input n, compute the Gödel code $q_n = \ulcorner \sigma_n \urcorner$. For cost bounds $b = 0, 1, 2, \ldots$, effectively list the finite set

$$E_b = \{p : c(p) = b\}.$$

Examine the elements of E_b in increasing numerical order. For each candidate p, compute $V(p, q_n)$. If the verifier returns 1, output p and halt.

Every operation at every finite stage is computable. Hence the procedure defines a partial computable function.

Suppose first that $T \vdash \sigma_n$. Then at least one valid proof code exists. The set of its costs is a nonempty subset of $\mathbb{N}$, so it has a least element b_0. The finite set E_{b_0} contains at least one proof of σ_n. The algorithm eventually reaches b_0, checks every earlier candidate, finds the least valid code in E_{b_0}, and halts. No proof of lower cost exists by minimality of b_0, and the numerical ordering resolves ties. Thus the returned proof is canonical and c-shortest.

Conversely, if the algorithm halts with output p, then $V(p, q_n) = 1$. By the meaning of the verifier, p is a valid T-proof of σ_n, so $T \vdash \sigma_n$.

Therefore the algorithm halts exactly on theorem indices, and its domain is the displayed set. □

Corollary 5.3 (Partial shortest-proof length)**.** *The function*

$$h_{T,V,c}(n) = c(H_{T,V,c}(n))$$

is partial computable, has the same domain as $H_{T,V,c}$, and returns the minimum proof cost of σ_n whenever σ_n is a theorem.

Corollary 5.4 (ZFC form)**.** *Fix a standard effective proof calculus for ZFC, a decidable proof verifier, an effective enumeration $\sigma_0, \sigma_1, \ldots$ of set-theoretic sentences, and an effectively proper*

proof-cost measure. Then

$$H_{\mathrm{ZFC}}(n) = \begin{cases} \textit{a canonical shortest ZFC proof of } \sigma_n, & \mathrm{ZFC} \vdash \sigma_n, \\ \uparrow, & \mathrm{ZFC} \nvdash \sigma_n \end{cases}$$

is a partial computable function.

Proof. A conventional syntactic presentation of ZFC has decidable formula parsing and mechanically checkable finite proofs. Apply Theorem 5.2. □

5.2 Why the theorem is both powerful and classical

The result says something genuinely universal:

Every theorem of a fixed effective theory has an algorithmically recoverable shortest proof.

But "shortest" is relative to the fixed presentation and proper cost measure, and "recoverable" means partial computability rather than practical feasibility.

The theorem is a direct consequence of the recursive enumerability of formal proofs [10, 11, 12]. Its purpose here is not to rename a classical observation and claim priority. Its purpose is to expose the exact positive architecture that survives undecidability and then connect that architecture to Tenneson's SIC and ATP program.

5.3 A pseudocode formulation

Listing 5.1: Pseudocode for the Universal Proof-Horizon Operator.

```
def proof_horizon(n):
    target = nth_sentence(n)
    for cost in natural_numbers():
        for proof_code in proof_codes_of_exact_cost(cost):
            if verify(proof_code, target):
                return proof_code
    # No negative return is licensed. The computation may continue forever.
```

The final comment is mathematically essential. Silence is not a disproof.

Chapter 6

The Church–Gödel Boundary

6.1 Why this does not decide ZFC

A total decision procedure would halt on every sentence and answer whether it is a ZFC theorem. The Proof-Horizon Operator does not do this. It gives a positive certificate when one exists and otherwise continues searching.

Proposition 6.1 (No total negative completion)**.** *Let T be a sufficiently expressive, consistent, effectively axiomatized theory with undecidable theorem set. There is no total computable function D satisfying*

$$D(n) = \begin{cases} 1, & T \vdash \sigma_n, \\ 0, & T \nvdash \sigma_n. \end{cases}$$

Proof. Such a function would decide the theorem set of T, contradicting Church's undecidability result for sufficiently strong effective theories [10]. □

The contrast can be stated in one line:

$$\begin{aligned} \text{partial theorem-to-shortest-proof operator} &: \text{exists}, \\ \text{total theorem/non-theorem decider} &: \text{does not exist in general.} \end{aligned}$$

6.2 Incompleteness is a different limitation

Gödel's incompleteness theorem says that every consistent, effectively axiomatized theory strong enough to formalize elementary arithmetic is incomplete [13]. Thus there are sentences for which neither the sentence nor its negation is provable in the theory (with the exact hypothesis depending on the version of the theorem). For such a sentence, both ordinary proof searches can run forever.

This is not a defect in the Proof-Horizon Operator. The operator's domain is theoremhood in the selected theory, not mathematical truth in every possible foundation. It ranges over all ZFC theorems, not all mathematics, not all truths about sets, and not the theorems of alternative foundations unless those systems are separately supplied.

6.3 What "all of ZFC mathematics" can responsibly mean

The phrase can mean:

> Every syntactically expressible ZFC sentence can be indexed, and every ZFC theorem lies in the halting domain of a single uniform proof-recovery procedure.

It cannot mean that every sentence is decided, that every mathematical truth is a ZFC theorem, that the resulting search is feasible, or that proof length is independent of presentation.

Chapter 7

Line-Shortest Proofs and the Finite-Branching Condition

The most delicate correctness issue in the entire argument is the word "shortest." If cost means total encoded proof size, Theorem 5.2 applies immediately because there are finitely many bit strings below any fixed size. If cost means number of proof lines, there may be infinitely many syntactically different one-line candidates. One cannot literally finish inspecting every one-line object before beginning the two-line search.

Theorem 7.1 (Shortest inference path under effective finite branching)**.** *Let G be an effectively generated directed proof-state graph with a designated initial state and target set. Suppose every vertex has a finite, effectively listable set of outgoing edges and each edge has unit cost. Then breadth-first search halts exactly when a target is reachable, and when it halts its predecessor map reconstructs a proof using the minimum number of edges.*

Proof. Breadth-first search explores vertices in nondecreasing graph distance from the initial state. Effective finite branching makes every finite depth layer finite and computable. Therefore the first target dequeued has minimum distance. Recording the predecessor vertex and edge label for every new state reconstructs a shortest path, whose labels are the proof steps. □

MIU satisfies these conditions. Finite formal libraries such as a frozen Metamath database can also support effectively finite search conditions once the admissible assertions, substitutions, and side conditions are fixed. Full ZFC proof search by raw line count needs an explicitly stated proof formalism ensuring finite branching, or else a proper size measure should be used.

Caution 7.2 (The correction that protects the headline). The universally valid computability claim is shortest proof under an effectively proper computable cost. "Fewest lines" is a valid corollary only under additional search-space hypotheses. This monograph does not hide that distinction.

Chapter 8

SICs and the Proof Horizon

Tenneson's *Depths of a Simulation* defines a semi-ideal computer (SIC) as a monotone stage process with an absorbing halting signal and then constructs canonical theorem-enumerating and target-search SICs [14]. The Proof-Horizon Theorem states, in that framework, that a canonical breadth-first target-search SIC halts exactly when the target is derivable and that its first halting stage is the target's shortest proof length in the selected formal model.

The universal operator of Theorem 5.2 and the SIC theorem illuminate one another:

- the universal operator supplies an explicitly computable semidecision procedure under a proper proof-cost measure;
- the SIC formulation turns proof length into a stage or horizon observable;
- the finite-branching graph theorem identifies when that horizon is realized by ordinary breadth-first search;
- learned policies can then be evaluated against the canonical horizon without being confused with soundness.

8.1 Theorem graphs

A theorem graph treats proof states as vertices and legal inference applications as labeled edges. In an unweighted graph, a proof is a path and a shortest proof is a shortest path. A predecessor map is therefore not bookkeeping external to the mathematics; it is the proof certificate itself.

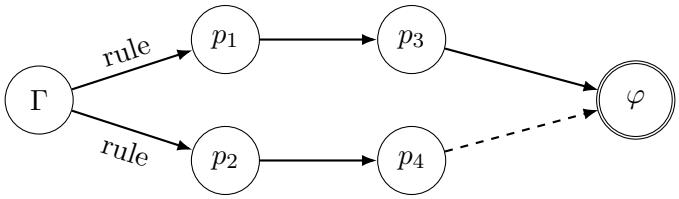

Figure 8.1: A proof-state graph. Breadth-first search gives a shortest edge path under finite branching; a learned policy changes the order of exploration but not what counts as a valid edge.

8.2 Search policy is not logical consequence

A learned model may rank actions, predict useful lemmas, or estimate distance to completion. None of these scores establishes theoremhood. Only the proof checker does. This separation is central both to Tenneson's ML–SIC–ATP account [15] and to the Predator engineering protocol.

Chapter 9

From Exhaustive Search to Navigation

The universal algorithm is theoretically sufficient and practically disastrous. It may spend nearly all of its resources on regions of theorem space irrelevant to the target. A331536 shows this problem in miniature. The role of machine learning and formalized creativity is not to alter the definition of proof but to alter the order in which legal possibilities are explored.

9.1 Three levels of creativity

Creativity can enter an ATP architecture at several levels:

1. **Inside a subordinate prover:** exploratory temperatures, novelty rewards, rarity preferences, lemma seeking, or stochastic branching.
2. **Inside an orchestrator:** choosing engines, changing representations, decomposing a target, allocating budgets, or transferring verified lemmas.
3. **Inside human–machine collaboration:** proposing analogies, translating between mathematical vocabularies, identifying overlooked sources, and revising the research question.

The verifier remains outside these creative choices. Creativity generates candidates; certification decides whether they count.

9.2 Predator as a case study

Predator is a versioned experimental ATP program developed within Tenneson's project. Its significance here is not that it already solves arbitrary mathematics, but that it operationalizes the transition from theorem enumeration to policy-guided search.

A preliminary Predator 8.004 calibration record dated August 3, 2026 reports two independently verified proofs of the Metamath theorem `prcom` in a frozen formal-proof environment [16]: one after 295 expansions using the dense-uniform model and one after 454 expansions using the theorem-balanced model. Both certificates passed in-process and external verification [17]. The earlier Predator 8.002 record required 2,633 expansions on the same calibration target. These numbers are promising engineering observations, not yet a general performance theorem.

The architecture illustrates the central separation:

$$\begin{aligned} \text{training and policy} &\longrightarrow \text{which legal state to examine next,} \\ \text{symbolic engine} &\longrightarrow \text{which actions are actually legal,} \\ \text{certificate verifier} &\longrightarrow \text{whether the claimed proof is valid.} \end{aligned}$$

9.3 Why analogy matters to proof navigation

A proof search system often needs to recognize that a new goal resembles an old one in a structurally useful way. The relevant shared "tag" may be a normal form, an invariant, a theorem family, a type signature, a dependency pattern, or a semantic domain such as semigroup theory. The hard problem is not merely attaching labels. It is learning which similarities preserve useful proof actions and which are superficial.

This is where an LLM can be valuable without being mistaken for a verifier. It can propose the comparison: "this target resembles a closure theorem," "this obstacle resembles an invariant argument," or "this formal statement may need a representation change." The ATP and checker must still construct and certify the path.

Chapter 10

Automated Logical Deciders

Tenneson's conjecture-settling and ALD manuscripts replace one-sided proving with symmetric logical investigation [18, 19]. The primitive architecture dovetails two proof-horizon computations:

$$H_T(\ulcorner \varphi \urcorner) \qquad \text{and} \qquad H_T(\ulcorner \neg\varphi \urcorner).$$

Corollary 10.1 (Primitive symmetric settlement)**.** *There is a partial procedure that, on input φ:*

(a) returns THEOREM *with a certificate if it finds a T-proof of φ;*

(b) returns REFUTABLE *with a certificate if it finds a T-proof of $\neg\varphi$;*

(c) otherwise continues searching.

If T is consistent, it cannot return both certified outcomes.

Proof. Dovetail the two partial proof searches, taking one finite step of each in alternation. If either finds a certificate, verify and return the corresponding result. Consistency rules out proofs of both φ and $\neg\varphi$. □

10.1 Independence is not prolonged silence

An independence claim requires a metamathematical certificate: for example, formal models of $T+\varphi$ and $T+\neg\varphi$, a relative consistency argument, a forcing construction, or another verifier-checkable witness appropriate to the formal setting. Running both proof searches for a long time establishes only bounded failure.

A mature ALD may therefore have heterogeneous subordinate engines:

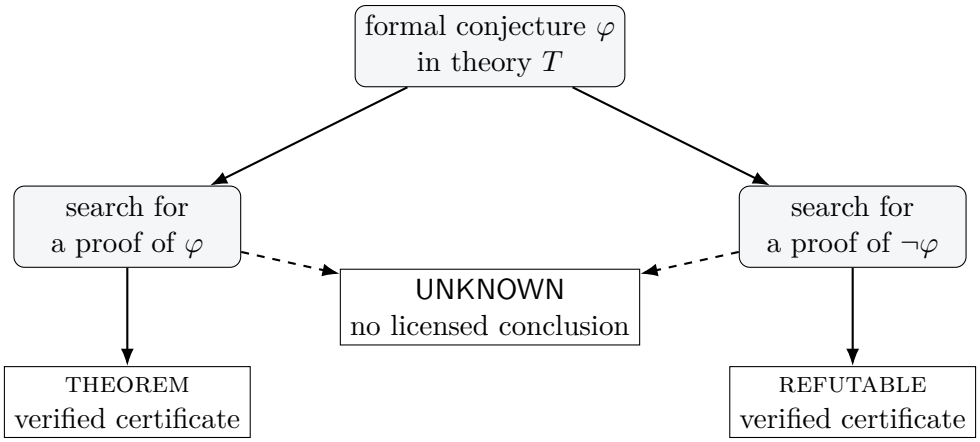

Figure 10.1: The primitive two-directional ALD. Failure to obtain a certificate is reported operationally as UNKNOWN, not logically as false or independent.

- conventional ATPs;
- SAT and SMT solvers;
- model finders and counterexample search;
- specialized decision procedures;
- Predator-like creative or learned provers;
- metamathematical engines for independence certificates;
- independent certificate verifiers.

The engines need not all be creative. The system is strongest when it uses the cheapest complete method available and reserves exploratory machinery for the regions where routine methods fail.

Chapter 11

What Has and Has Not Been "Figured Out"

11.1 What has been established

The following claims are supported by the proof in this monograph.

1. The well-formed formulas and sentences of ZFC can be effectively enumerated after a coding convention is fixed.
2. Every ZFC theorem has at least one finite formal proof in the fixed calculus.
3. Under an effectively proper computable proof-cost measure, a partial computable operator returns a canonical minimum-cost proof of every ZFC theorem.
4. The operator halts exactly on theorem indices.
5. Symmetric dovetailing produces a sound partial classifier into theorem or refutable, with certificates.

11.2 What has not been established

The monograph does not show any of the following.

1. It does not give a total decision procedure for ZFC.
2. It does not determine whether a silent computation corresponds to falsehood, independence, or insufficient resources.

3. It does not make exhaustive proof search computationally feasible.
4. It does not establish an absolute notion of shortest proof.
5. It does not settle an open mathematical conjecture.
6. It does not show that Predator is generally superior to established ATP systems.
7. It does not claim the underlying recursive-enumeration theorem as historically new.

The discovery in this work is therefore best described as a recovered conceptual spine. A question asked in 2016, an MIU proof algorithm, an OEIS growth sequence, a SIC theorem, and a modern ATP experiment are recognized as parts of one program:

$$\text{universal proof existence by enumeration} \quad + \quad \text{intelligent navigation of the enumeration.}$$

Chapter 12

A Research Program

12.1 Formalization priorities

The next mathematical steps are concrete.

1. Formalize Theorem 5.2 in a proof assistant or Metamath environment, including the effective-properness condition.
2. Compare proof-cost measures: encoded bit length, logical inference count, raw proof-file steps, and weighted transaction cost.
3. Identify proof systems in which line-count layers are effectively finite and the BFS corollary applies exactly.
4. Define contamination-safe benchmark protocols for learned proof-horizon approximation.
5. Test positive controls, negative controls, paired $\varphi/\neg\varphi$ controls, equivalent reformulations, and known independence controls.

12.2 The HaloProof gauntlet

HaloProof is a proposed public benchmark: using the framework of Goldblatt's *Lectures on the Hyperreals*, construct and formally verify a proof that the halo of zero is equipollent with the real numbers [20, 21]. Goldblatt's framework is part of the benchmark specification. The challenge does not claim that a proof route is shortest, nor that all necessary ideas are presented explicitly in the book.

HaloProof matters here because it separates the universal existence theorem from the practical discovery problem. The Proof-Horizon Operator guarantees only that a formal proof, if it exists in

the chosen theory, is somewhere in the search space. The benchmark asks whether an actual ATP can find and certify it under a transparent protocol.

12.3 From known theorems to genuine discovery

A responsible progression is:

$$\begin{aligned} \text{known theorems} &\longrightarrow \text{withheld benchmarks} \\ &\longrightarrow \text{paired theorem/refutation controls} \\ &\longrightarrow \text{modest open conjectures} \\ &\longrightarrow \text{substantial conjecture settlement.} \end{aligned}$$

The first new conjecture need not be famous. It must be genuinely open before the run, precisely formalized, contamination-audited, certified, independently checked, and reproducible.

Chapter 13

Conclusion: The Dream and the Boundary

The 2016 dream was not that a computer would merely print every consequence of a formal system. It was that a machine might navigate proof space with something resembling cognition. The recovered theorem does not complete that dream, but it clarifies the ground beneath it.

For every theorem in an effective formal system, a canonical shortest proof is recoverable by a partial algorithm once the proof presentation and a proper cost measure are fixed. Church's theorem does not forbid that machine. It forbids the stronger machine that always halts and correctly announces that no proof exists. Gödel's theorem adds that no single sufficiently strong effective theory captures every mathematical truth.

The theoretical baseline is therefore complete enough to be useful:

proof search can be universal on theorems.

The practical problem remains enormous:

how should the machine know where to look?

Matos answered that question for MIU by exploiting the special structure of the system. A331536 showed how rapidly the space expands. SICs turned proof length into a horizon. Predator attempts to learn the terrain. ALDs organize several forms of search without confusing silence with knowledge. And an LLM — the kind of machine imagined in a much earlier lecture — helped its human collaborator recognize that these were not disconnected projects after all.

Appendix A

Executable MIU Demonstration

The companion Python program verifies the first eight values of A331536 and uses breadth-first search to reconstruct a shortest MIU derivation under unit rule cost. It is deliberately small enough to inspect. It does not pretend to implement the universal ZFC search, whose proof database and parser would be vastly larger.

```
#!/usr/bin/env python3
"""Small executable companion to The Universal Proof-Horizon Operator.

This file does two things:
1. It implements breadth-first search in Hofstadter's MIU system, returning
   a shortest derivation because each edge has unit cost.
2. It verifies the initial values of OEIS A331536 by counting all distinct
   MIU theorems reachable within a bounded number of rule applications.

It is a finite demonstration, not an implementation of ZFC proof search.
"""

from __future__ import annotations

from collections import deque
from dataclasses import dataclass
from typing import Iterable, Iterator


@dataclass(frozen=True)
class Step:
    source: str
    target: str
    rule: str


def _replace_each(word: str, old: str, new: str) -> Iterator[str]:
```

```
    """Yield the result of replacing each occurrence separately."""
    start = 0
    while True:
        index = word.find(old, start)
        if index < 0:
            return
        yield word[:index] + new + word[index + len(old) :]
        start = index + 1


def miu_successors(word: str) -> Iterable[tuple[str, str]]:
    """Generate all one-step MIU successors, with rule labels."""
    if not word.startswith("M"):
        raise ValueError(f"Not an MIU well-formed formula: {word!r}")

    results: list[tuple[str, str]] = []

    # Rule I: xI -> xIU.
    if word.endswith("I"):
        results.append((word + "U", "I: xI -> xIU"))

    # Rule II: Mx -> Mxx.
    tail = word[1:]
    results.append(("M" + tail + tail, "II: Mx -> Mxx"))

    # Rule III: xIIIy -> xUy, one occurrence at a time.
    results.extend((candidate, "III: xIIIy -> xUy") for candidate in _replace_each(word, "III", "U")
        )

    # Rule IV: xUUy -> xy, one occurrence at a time.
    results.extend((candidate, "IV: xUUy -> xy") for candidate in _replace_each(word, "UU", ""))

    # Preserve deterministic order while removing duplicates.
    return tuple(dict.fromkeys(results))


def shortest_miu_proof(target: str, max_depth: int | None = None) -> list[Step] | None:
    """Return a shortest MIU proof of target, or None within max_depth.

    With max_depth=None, this semidecision procedure halts on MIU theorems
    and may run forever on non-theorems.
    """
    if target == "MI":
        return []

    queue: deque[tuple[str, int]] = deque([("MI", 0)])
    parent: dict[str, tuple[str, str] | None] = {"MI": None}

```

```
    while queue:
        current, depth = queue.popleft()
        if max_depth is not None and depth >= max_depth:
            continue

        for successor, rule in miu_successors(current):
            if successor in parent:
                continue
            parent[successor] = (current, rule)
            if successor == target:
                chain: list[Step] = []
                cursor = successor
                while parent[cursor] is not None:
                    source, used_rule = parent[cursor] # type: ignore[misc]
                    chain.append(Step(source, cursor, used_rule))
                    cursor = source
                chain.reverse()
                return chain
            queue.append((successor, depth + 1))

    return None


def cumulative_miu_counts(max_rule_steps: int) -> list[int]:
    """Count distinct MIU theorems reachable in at most d steps, d=0..max."""
    reached = {"MI"}
    frontier = {"MI"}
    counts = [1]

    for _ in range(max_rule_steps):
        next_frontier = {
            successor
            for word in frontier
            for successor, _ in miu_successors(word)
            if successor not in reached
        }
        reached.update(next_frontier)
        frontier = next_frontier
        counts.append(len(reached))
    return counts


def main() -> None:
    # OEIS A331536 uses a(1) for zero rule steps, a(2) for at most one, etc.
    expected = [1, 3, 6, 11, 25, 69, 282, 1730]
    observed = cumulative_miu_counts(len(expected) - 1)
    assert observed == expected, (observed, expected)

```

```
    print("Verified initial A331536 values:", observed)

    target = "MUIU"
    proof = shortest_miu_proof(target, max_depth=12)
    if proof is None:
        print(f"No proof of {target} found within the finite demonstration budget.")
        return

    print(f"Shortest proof of {target} has {len(proof)} rule applications:")
    print("MI")
    for step in proof:
        print(f" -> {step.target:<18} [{step.rule}]")


if __name__ == "__main__":
    main()
```

Appendix B

Audit Notes

B.1 Mathematical audit

Three correctness passes were applied to the main theorem.

1. **Computability pass.** Every operation in the algorithm is total computable at each finite stage: sentence enumeration, cost-layer generation, and certificate verification.
2. **Minimality pass.** The proof-cost measure is required to have effectively finite sublevel sets. This closes the gap that would arise from casually ordering an infinite set of one-line proof candidates.
3. **Boundary pass.** Divergence is explicitly separated from refutation and independence. The theorem is stated as a semidecision result, not a decision result.

B.2 Citation audit

The bibliography is limited to sources used directly in the body: Hofstadter on MIU and analogy; Howard on proofs and programs; Matos and Antunes on MIU proof algorithms and proof length; OEIS A331536; the 2016 lecture; Tenneson's SIC, ML–SIC, ALD, conjecture-settling, HaloProof, and Predator records; Church, Turing, Gödel, and Rogers for computability and undecidability; Cook and Reckhow for proof-system relativity; Metamath documentation; and Goldblatt for the HaloProof source framework.

B.3 Authorship statement for reuse

Brian Tenneson originated the research question and developed the core synthesis. AI assistance helped recover, organize, test, and express the argument. The central connection among Matos's MIU work, effective enumeration of ZFC formulas, proof horizons, Predator, and automated logical deciders came from Tenneson; the AI's role was collaborative analysis, mathematical criticism, source organization, drafting, and editorial refinement.

References

[1] Brian Tenneson. *Proof Theory and Automated Theorem Proving*. Lecture presented to graduate mathematics students at the University of Montana; video recording. 2016. URL: https://www.youtube.com/watch?v=PCflygaCLXc.

[2] Douglas R. Hofstadter. *Gödel, Escher, Bach: An Eternal Golden Braid*. The MIU system is introduced on pp. 33–41. Basic Books, 1979.

[3] Douglas Hofstadter and Emmanuel Sander. *Surfaces and Essences: Analogy as the Fuel and Fire of Thinking*. Basic Books, 2013.

[4] William A. Howard. "The Formulae-as-Types Notion of Construction". In: *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*. Ed. by J. P. Seldin and J. R. Hindley. Manuscript circulated in 1969. Academic Press, 1980, pp. 479–490.

[5] Armando B. Matos. *The Theorems of the Formal System MIU*. Tech. rep. DCC-97-08. Departamento de Ciência de Computadores, Universidade do Porto, 1997.

[6] Armando B. Matos and Luís Filipe Antunes. *Short Proofs for MIU Theorems*. Tech. rep. DCC-98-01. Departamento de Ciência de Computadores, Universidade do Porto, 1998.

[7] Armando B. Matos. *On the Number of Lines of Theorems in the Formal System MIU*. Tech. rep. Faculdade de Ciências and Laboratório de Inteligência Artificial e Ciência de Computadores, Universidade do Porto, 2000, pp. 1–10.

[8] Stephen A. Cook and Robert A. Reckhow. "The Relative Efficiency of Propositional Proof Systems". In: *Journal of Symbolic Logic* 44.1 (1979), pp. 36–50. DOI: 10.2307/2273702.

[9] Brian Tenneson and contributors. *A331536: Number of Theorems in the MIU Formal System Which Can Be Proved in n Steps or Fewer Starting with the Axiom MI*. The On-Line Encyclopedia of Integer Sequences. Created Jan. 19, 2020; later terms contributed by Rémy Sigrist, Sean A. Irvine, and Martin Fuller. 2020. URL: https://oeis.org/A331536.

[10] Alonzo Church. "An Unsolvable Problem of Elementary Number Theory". In: *American Journal of Mathematics* 58.2 (1936), pp. 345–363. DOI: 10.2307/2371045.

[11] Alan M. Turing. "On Computable Numbers, with an Application to the Entscheidungsproblem". In: *Proceedings of the London Mathematical Society*. 2nd ser. 42 (1936), pp. 230–265.

[12] Jr. Hartley Rogers. *Theory of Recursive Functions and Effective Computability*. MIT Press, 1987.

[13] Kurt Gödel. "Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I". In: *Monatshefte für Mathematik und Physik* 38 (1931), pp. 173–198. DOI: 10.1007/BF01700692.

[14] Brian Tenneson. *Depths of a Simulation: Computers, Formal Systems, Simulations, and Automated Theorem Proving*. Self-published research manuscript, Version 41 series. Contains the SIC framework and the Proof-Horizon Theorem. 2026.

[15] Brian Tenneson. *ML–SIC–ATP Theory: Learned Search Policies for Proof Search*. Self-published research manuscript, Version 1.1. July 2026.

[16] Norman D. Megill and David A. Wheeler. *Metamath: A Computer Language for Mathematical Proofs*. 2nd ed. Lulu Press, 2019. URL: https://us.metamath.org/downloads/metamath.pdf.

[17] Brian Tenneson. *Predator 8.004 Experiment Record: prcom Calibration and sgrpcl Preprocessing*. Unpublished reproducibility log. Run record dated Aug. 3, 2026. Aug. 2026.

[18] Brian Tenneson. *Conjecture Settling, Part I*. Independent research manuscript. 2026.

[19] Brian Tenneson. *Automated Logical Deciders: Certificate Halting, Shared Lemma Pools, and Machine-Learning Guidance*. Independent research manuscript. Aug. 2026.

[20] Robert Goldblatt. *Lectures on the Hyperreals: An Introduction to Nonstandard Analysis*. Vol. 188. Graduate Texts in Mathematics. Springer, 1998. DOI: 10.1007/978-1-4612-0615-6.

[21] Brian Tenneson. *Machine-Learned ATP Benchmark Blueprint: The Halo of Zero Equipollent to the Reals*. Independent research manuscript. 2026.